

Exp No:9 MONITORING AND LOGGING FABRIC NETWORKS

AIM

To monitor the Hyperledger Fabric network by inspecting Docker containers, viewing peer and orderer logs, checking network status, monitoring resource utilization, and analyzing blockchain transaction logs for troubleshooting and performance analysis.

PROCEDURE

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

Stop any existing Fabric network and start a fresh network with the application channel and Certificate Authorities.

```
./network.sh down
```

```
./network.sh up createChannel -ca
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ cd ~/fabric-dev/fabric-samples/test-network
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb with crypto from 'Certificate Authorities'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
Generating certificates using Fabric CA
```

Step 3: Verify Running Containers

Display all active Docker containers.

```
docker ps
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ docker ps
CONTAINER ID   IMAGE                                COMMAND
CREATED        STATUS      PORTS
NAMES
a3ff043b8c62   hyperledger/fabric-orderer:latest   "orderer"
About a minute ago Up About a minute 0.0.0.0:7050->7050/tcp, [::]:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053->7053/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp
orderer.example.com
18ad1e0dcba0   hyperledger/fabric-peer:latest      "peer node start"
About a minute ago Up About a minute 0.0.0.0:9051->9051/tcp, [::]:9051->9051/tcp, 0.0.0.0:9445->9445/tcp, [::]:9445->9445/tcp
peer0.org2.example.com
eae02888dc5a   hyperledger/fabric-peer:latest      "peer node start"
About a minute ago Up About a minute 0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444->9444/tcp
peer0.org1.example.com
7d1aa33bbc74   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se..."
About a minute ago Up About a minute 0.0.0.0:8054->8054/tcp, [::]:8054->8054/tcp, 0.0.0.0:18054->18054/tcp, [::]:18054->18054/tcp
ca_org2
7396d41d5681   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se..."
About a minute ago Up About a minute 0.0.0.0:9054->9054/tcp, [::]:9054->9054/tcp, 0.0.0.0:19054->19054/tcp, [::]:19054->19054/tcp
ca_orderer
b34d3aad706c   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se..."
About a minute ago Up About a minute 0.0.0.0:7054->7054/tcp, [::]:7054->7054/tcp, 0.0.0.0:17054->17054/tcp, [::]:17054->17054/tcp
ca_org1
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 4: Monitor CPU and Memory Usage

Display live CPU and memory utilization of the Fabric containers. Allow the statistics to run for a few seconds and press Ctrl+C to stop.

```
docker stats
```

```

":'["['{'"host":'peer0.org2.example.com','port":'9051'}'],''],
{'"version":'0','MSP':{'"mod_policy":'','"value":'null
{'"version":'0','"version":'1'}'],'"mod_policy":'
{'"policies":'{}','"values":'{}','"version":'0','"mod_
CONTAINER ID    NAME                                CPU %       MEM USAGE / LIMIT     MEM
%      NET I/O      BLOCK I/O    PIDS
a3ff043b8c62    orderer.example.com              0.21%       19.39MiB / 7.479GiB    0.25
%      58.2kB / 209kB    33.3MB / 193kB    13
18ad1e0dcba0    peer0.org2.example.com           3.96%       31.41MiB / 7.479GiB    0.41
%      283kB / 216kB     41kB / 393kB     15
eae02888dc5a    peer0.org1.example.com           3.55%       37.36MiB / 7.479GiB    0.49
%      283kB / 216kB     65.5kB / 393kB   14
7d1aa33bbc74    ca_org2                           0.00%       9.887MiB / 7.479GiB    0.13
%      29.4kB / 44.2kB   24.6kB / 737kB    12
7396d41d5681    ca_orderer                        0.00%       10.71MiB / 7.479GiB    0.14
%      52.7kB / 84.7kB   26.1MB / 1.05MB   11
b34d3aad706c    ca_org1                           0.00%       9.707MiB / 7.479GiB    0.13
%      31.8kB / 49.9kB   4.22MB / 737kB    11

```

For a single snapshot:

```
docker stats --no-stream
```

```

naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ docker stat
s --no-stream
CONTAINER ID    NAME                                CPU %       MEM USAGE / LIMIT     MEM
%      NET I/O      BLOCK I/O    PIDS
a3ff043b8c62    orderer.example.com              0.15%       19.63MiB / 7.479GiB    0.26
%      58.6kB / 209kB    33.3MB / 193kB    13
18ad1e0dcba0    peer0.org2.example.com           3.90%       31.35MiB / 7.479GiB    0.41
%      311kB / 244kB     41kB / 393kB     15
eae02888dc5a    peer0.org1.example.com           3.76%       37.23MiB / 7.479GiB    0.49
%      311kB / 244kB     65.5kB / 393kB   14
7d1aa33bbc74    ca_org2                           0.04%       9.387MiB / 7.479GiB    0.12
%      29.4kB / 44.2kB   24.6kB / 737kB    12
7396d41d5681    ca_orderer                        0.00%       10.95MiB / 7.479GiB    0.14
%      52.7kB / 84.7kB   26.1MB / 1.05MB   11
b34d3aad706c    ca_org1                           0.00%       10.21MiB / 7.479GiB    0.13
%      31.8kB / 49.9kB   4.22MB / 737kB    11
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$

```

Step 5: View Peer Logs

Display the recent logs of the Org1 peer.


```
docker logs --tail 50 peer0.org1.example.com
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ docker logs
--tail 50 peer0.org1.example.com
2026-09-01 18:58:31.940 UTC 001c INFO [discovery] NewService -> Created with
config TLS: true, authCacheMaxSize: 1000, authCachePurgeRatio: 0.750000
2026-09-01 18:58:31.940 UTC 001d INFO [nodeCmd] serve -> Discovery service a
ctivated
2026-09-01 18:58:31.940 UTC 001e INFO [nodeCmd] serve -> Starting peer with
Gateway enabled
2026-09-01 18:58:31.940 UTC 001f INFO [nodeCmd] serve -> Starting peer with
ID=[peer0.org1.example.com], network ID=[dev], address=[peer0.org1.example.c
om:7051]
2026-09-01 18:58:31.940 UTC 0020 INFO [nodeCmd] serve -> Started peer with I
D=[peer0.org1.example.com], network ID=[dev], address=[peer0.org1.example.co
m:7051]
2026-09-01 18:58:31.940 UTC 0021 INFO [kvledger] LoadPreResetHeight -> Loadi
ng prereset height from path [/var/hyperledger/production/ledgersData/chains
]
2026-09-01 18:58:31.940 UTC 0022 INFO [blkstorage] preResetHtFiles -> No act
ive channels passed
2026-09-01 18:58:38.045 UTC 0023 INFO [ledgermgmt] CreateLedger -> Creating
ledger [mychannel] with genesis block
2026-09-01 18:58:38.050 UTC 0024 INFO [blkstorage] newBlockfileMgr -> Gettin
g block information from block storage
2026-09-01 18:58:38.068 UTC 0025 INFO [kvledger] commit -> [mychannel] Commi
tted block [0] with 1 transaction(s) in 13ms (state_validation=0ms block_and
_pvtdata_commit=6ms state_commit=3ms) commitHash=[]
2026-09-01 18:58:38.068 UTC 0026 INFO [kvledger] updateLedgerStatus -> Updat
ing ledger [mychannel] status to [ACTIVE]
2026-09-01 18:58:38.070 UTC 0027 INFO [ledgermgmt] CreateLedger -> Created l
edger [mychannel] with genesis block
2026-09-01 18:58:38.072 UTC 0028 WARN [peer.orderers] Update -> Config defin
es both orderer org specific endpoints and global endpoints, global endpoint
s will be ignored channel=mychannel
2026-09-01 18:58:38.072 UTC 0029 INFO [gossip.gossip] JoinChan -> Joining go
ssip network of channel mychannel with 2 organizations
```

Step 6: View Orderer Logs

Display the recent logs of the ordering service.

```
docker logs --tail 50 orderer.example.com
```

```
naveen@LAPTOP-2581ZDTL:~/fabric-dev/fabric-samples/test-network$ docker logs
--tail 50 orderer.example.com
Version: v2.5.16
Commit SHA: f871cf9
Go version: go1.26.4
OS/Arch: linux/amd64
2026-09-01 18:58:31.823 UTC 000f INFO [orderer.common.server] Main -> Beginn
ing to serve requests
2026-09-01 18:58:34.936 UTC 0010 INFO [blkstorage] newBlockfileMgr -> Gettin
g block information from block storage
2026-09-01 18:58:34.956 UTC 0011 INFO [orderer.consensus.etcdraft] HandleCha
in -> EvictionSuspicion not set, defaulting to 10m0s
2026-09-01 18:58:34.956 UTC 0012 INFO [orderer.consensus.etcdraft] HandleCha
in -> Without system channel: after eviction Registrar.SwitchToFollower will
be called
2026-09-01 18:58:34.956 UTC 0013 INFO [orderer.consensus.etcdraft] createOrR
eadWAL -> No WAL data found, creating new WAL at path '/var/hyperledger/prod
uction/orderer/etcdraft/wal/mychannel' channel=mychannel node=1
2026-09-01 18:58:34.965 UTC 0014 INFO [orderer.common.multichannel] createA
sMember -> Joining channel: {mychannel consenter active 1}
2026-09-01 18:58:34.968 UTC 0015 INFO [orderer.consensus.etcdraft] Start ->
Starting Raft node channel=mychannel node=1
2026-09-01 18:58:34.968 UTC 0016 INFO [orderer.common.cluster] Configure ->
Entering, channel: mychannel, nodes: []
2026-09-01 18:58:34.968 UTC 0017 INFO [orderer.common.cluster] Configure ->
Exiting
2026-09-01 18:58:34.968 UTC 0018 INFO [orderer.consensus.etcdraft] start ->
Starting raft node as part of a new channel channel=mychannel node=1
2026-09-01 18:58:34.968 UTC 0019 INFO [orderer.consensus.etcdraft] switchToC
onfig -> 1 switched to configuration voters=() channel=mychannel node=1
2026-09-01 18:58:34.968 UTC 001a INFO [orderer.consensus.etcdraft] becomeFol
lower -> 1 became follower at term 0 channel=mychannel node=1
2026-09-01 18:58:34.968 UTC 001b INFO [orderer.consensus.etcdraft] newRaft -
> newRaft 1 [peers: [], term: 0, commit: 0, applied: 0, lastindex: 0, lastte
```

Step 7: View Certificate Authority Logs

Display the logs of the Org1 Certificate Authority.

```
docker logs ca_org1
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ docker logs
ca_org1
2026/09/01 18:58:26 [DEBUG] Home directory: /etc/hyperledger/fabric-ca-server
2026/09/01 18:58:26 [INFO] Configuration file location: /etc/hyperledger/fab
ric-ca-server/fabric-ca-server-config.yaml
2026/09/01 18:58:26 [INFO] CA config with combined yaml file and environment
variable overrides (sensitive lines excluded):
address: 0.0.0.0
affiliations:
  org1:
    - department1
    - department2
  org2:
    - department1
bccsp:
  default: SW
  sw:
    filekeystore:
      keystore: msp/keystore
    hash: SHA2
    security: 256
ca:
  certfile: ca-cert.pem
  chainfile: ca-chain.pem
  keyfile: ""
  name: ca-org1
  reenrollignorecertexpiry: false
cacount: 0
cafiles: []
cfg:
  affiliations:
    allowremove: false
```

Step 8: Verify the Network Channel

Load the Org1 peer environment and list the channels joined by the peer.

```
source setOrg1.sh
```

```
peer channel list
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ source setO
rg1.sh
peer channel list
-bash: setOrg1.sh: No such file or directory
2026-09-01 19:05:27.434 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorse
r and orderer connections initialized
Channels peers has joined:
mychannel
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 9: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode to mychannel.

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-
basic/chaincode-javascript
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.s
h deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode
-javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-bas
ic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
```

Step 10: Query Installed Chaincode

Verify that the chaincode package has been installed on the peer.

```
peer lifecycle chaincode queryinstalled
chaincode installation is not required
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer lifecy
cle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cb
cc7f59b629, Label: basic_1.0
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ █
```

Step 11: Set TLS Environment Variables

Set the TLS certificates for Org1 and Org2.

```
export
ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.o
rg1.example.com/tls/ca.crt

export
ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.o
```



```
rg2.example.com/tls/ca.crt
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ export ORG1
_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org
1.example.com/tls/ca.crt

export ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peer
s/peer0.org2.example.com/tls/ca.crt
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 12: Initialize the Ledger

Initialize the ledger with the sample assets. Both Org1 and Org2 peers are included for endorsement.

```
peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $ORG1_TLS \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $ORG2_TLS \
-c '{"function":"InitLedger","Args":[]}'

naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $ORG1_TLS \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $ORG2_TLS \
-c '{"function":"InitLedger","Args":[]}'
2026-09-01 19:07:57.558 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery
```

Step 13: Monitor Blockchain Transactions

Query all assets stored in the ledger.

```
peer chaincode query -C mychannel -n basic \
-c '{"Args":["GetAllAssets"]}'

naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode query -C mychannel -n basic \
-c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":
5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owne
r":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green",
"ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValu
e":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"ass
et"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana",
"Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asse
t6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Step 14: View Recent Docker Events

Monitor Docker events generated by the Fabric network during the previous ten minutes. Press Ctrl+C after capturing the required events.

```
docker events --since 10m
```

For a fixed monitoring period:

```
timeout 15 docker events --since 10m
```

```
naveen@LAPTOP-25812DTL:~/fabric-dev/fabric-samples/test-network$ docker events --since 10m
2026-09-01T19:05:56.969660796Z container create 1bcb62fb0065c3d2b961df63726a81d2ee36289c21130d5d10f9444a867fe6e4 (desktop.docker.io/ports.scheme=v2, desktop.docker.io/wsl-distro=Ubuntu, image=hyperledger/fabric-nodeenv:2.5, name=hungry_dubinsky, org.opencontainers.image.created=2024-12-04T09:38:52.504Z, org.opencontainers.image.description=Hyperledger Fabric Node.js Smart Contracts, org.opencontainers.image.licenses=Apache-2.0, org.opencontainers.image.revision=b3720d0665328c8580a686b906012de3c8466c82, org.opencontainers.image.source=https://github.com/hyperledger/fabric-chaincode-node, org.opencontainers.image.title=fabric-chaincode-node, org.opencontainers.image.url=https://github.com/hyperledger/fabric-chaincode-node, org.opencontainers.image.version=2.5.8)
2026-09-01T19:05:56.992574676Z container extract-to-dir 1bcb62fb0065c3d2b961df63726a81d2ee36289c21130d5d10f9444a867fe6e4 (desktop.docker.io/ports.scheme=v2, desktop.docker.io/wsl-distro=Ubuntu, image=hyperledger/fabric-nodeenv:2.5, name=hungry_dubinsky, org.opencontainers.image.created=2024-12-04T09:38:52.504Z, org.opencontainers.image.description=Hyperledger Fabric Node.js Smart Contracts, org.opencontainers.image.licenses=Apache-2.0, org.opencontainers.image.revision=b3720d0665328c8580a686b906012de3c8466c82, org.opencontainers.image.source=https://github.com/hyperledger/fabric-chaincode-node, org.opencontainers.image.title=fabric-chaincode-node, org.opencontainers.image.url=https://github.com/hyperledger/fabric-chaincode-node, org.opencontainers.image.version=2.5.8)
```

Step 15: Stop the Fabric Network

```
./network.sh down
```

```
naveen@LAPTOP-25812DTL:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
h down
Using docker and docker-compose
Stopping network
[+] down 3/6
✓Container peer0.org1.example.com Removed 1.1s
✓Container peer0.org2.example.com Removed 1.0s
.. Container ca_orderer Stopping 1.6s
✓Container orderer.example.com Removed 0.5s
.. Container ca_org1 Stopping 1.6s
.. Container ca_org2 Stopping 1.6s
```

RESULT

The Hyperledger Fabric network was successfully monitored using Docker utilities and Fabric CLI commands. Peer nodes, the ordering service, Certificate Authorities, channels, chaincode installation, and blockchain ledger data were verified through monitoring commands and log analysis. The experiment demonstrated effective monitoring and troubleshooting techniques for maintaining a reliable Hyperledger Fabric network.